DoS Lab Environment Setup Guide

How to rebuild this lab from scratch and run it again

Goal: stand up nginx + Slowloris + Netdata inside a VM, run the attack, apply mitigation, and confirm the result.

0. Why Run This in a VM

Slowloris is offensive tooling and you point it at a live web server. Doing this in a disposable VM keeps it isolated: nothing touches your real machine, config changes are sandboxed, and you can revert a snapshot if anything breaks. Do not run this lab against your personal machine or any server you don't own.

Snapshot tip: Take a VM snapshot once the environment is fully set up (after Section 4). If a later run leaves the system in a weird state, revert to that snapshot instead of rebuilding.

1. Provision the VM

Any Linux VM with a desktop environment and RDP access works (the lab is built around an Ubuntu-based VM). You need a GUI because Netdata is viewed in a browser running inside the VM.

- **OS:** Ubuntu Desktop (or equivalent Debian-based distro)
- **Access:** RDP enabled (e.g. xrdp), or use the hypervisor's console
- **Network:** Internet access for package installs

2. Install & Verify nginx

```
sudo apt update
sudo apt install -y nginx
sudo systemctl enable --now nginx
```

`enable --now` both starts nginx immediately and sets it to launch on boot. Confirm it's up:

```
sudo systemctl status nginx
sudo ss -tlnp | grep :80
```

You should see `active (running)` and a process listening on port 80. Visiting `http://127.0.0.1` in the VM's browser should show the nginx welcome page.

3. Install the Slowloris Tool

Check whether `slowloris-run` is already present (the course VM ships with it):

```
which slowloris-run
```

If it returns a path, you're done. If not, the widely-used Python implementation installs via pip:

```
sudo apt install -y python3-pip
pip3 install slowloris
```

Note: the pip package is invoked as `slowloris <target>`. The course wrapper `slowloris-run` takes the target plus `-s` for socket count. Use whichever your environment provides; the concept is identical.

4. Install Netdata (Monitoring)

```
wget -O /tmp/netdata-kickstart.sh https://my-netdata.io/kickstart.sh \
  && sudo sh /tmp/netdata-kickstart.sh
```

The kickstart script auto-detects the OS, installs dependencies, and registers Netdata as a service on port 19999. Confirm it's listening:

```
sudo systemctl status netdata
sudo ss -tlnp | grep 19999
```

Accessing the dashboard

1. In the VM's Firefox, go to `http://127.0.0.1:19999`
2. Click *"Skip and use the dashboard anonymously"* to avoid creating an account.
3. Open the **Dashboard** tab and surface two charts: `net.networks` (Network Traffic) and `ip.tcpsock` (TCP Connections).

If Firefox can't connect to 127.0.0.1:19999

- Service not running → `sudo systemctl start netdata`
- Not installed → re-run the kickstart command and watch for errors
- Just installed → wait 10–20s for it to bind, then `sudo ss -tlnp | grep 19999`
- Make sure the address bar has the full `http://` prefix, or Firefox treats it as a search
- Confirm you're using the VM's Firefox, since `127.0.0.1` only reaches Netdata from the same machine it's running on.

5. Run the Lab

Attack 1 — Unprotected baseline

```
slowloris-run 127.0.0.1 -s 500
```

Watch the TCP Connections chart climb and hold high. Let it run 30+ seconds, screenshot, then `Ctrl+C`.

Apply mitigation

Back up first:

```
sudo cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak
```

Add to the `http { }` block in `/etc/nginx/nginx.conf`, but edit the existing **keepalive_timeout** rather than adding a second one (a duplicate fails the config test):

```
limit_conn_zone $binary_remote_addr zone=addr:10m;
client_header_timeout 5s;
client_body_timeout   5s;
send_timeout          5s;
keepalive_timeout     5;
```

Add to the `server { }` block in `/etc/nginx/sites-available/default`:

```
limit_conn addr 10;
```

Check for an existing keepalive line before saving:

```
grep -n keepalive_timeout /etc/nginx/nginx.conf
```

Then validate and apply:

```
sudo nginx -t && sudo systemctl reload nginx
```

Look for `syntax is ok` and `test is successful` before the reload runs.

Attack 2 — Protected

```
slowloris-run 127.0.0.1 -s 500
```

Run the identical attack. This time connections should spike briefly and collapse back to baseline instead of holding. Let it run 30+ seconds, screenshot, then `Ctrl+C`.

6. Reset Between Runs

To return to the unprotected state and run the comparison again:

```
sudo cp /etc/nginx/nginx.conf.bak /etc/nginx/nginx.conf
sudo nginx -t && sudo systemctl reload nginx
```

This restores your backup. You don't need to reset Netdata between attacks, since its graphs are continuous and scroll automatically.

7. Stretch — pcap Analysis

From the home directory so paths stay consistent:

```
cd ~
wget
https://raw.githubusercontent.com/codepath/cyb102-file-storage/main/project4_pcap_files.zip
unzip project4_pcap_files.zip
```

Open each capture in Wireshark and compare TCP flags. The **vulnerable** server shows many connections established and held open; the **mitigated** server shows the server sending RST packets to tear down slow connections quickly. Filter with `tcp.flags.reset == 1` to count resets in each file as evidence.

8. Quick Checklist

- VM up, RDP working at a usable resolution
- nginx running, welcome page loads at `127.0.0.1`
- `slowloris-run` (or `slowloris`) installed
- Netdata reachable at `127.0.0.1:19999`, TCP Connections chart visible
- Config backed up before editing
- Mitigation directives added, `nginx -t` passes
- Before/after screenshots captured on the same K-conns scale